

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Assignment 2: Performance Evaluation of OpenMP Programs using Parallel Performance Metrics

Learning Objectives

After completing this assignment, students will be able to:

- Implement basic OpenMP parallel programs.
- Measure execution time using `omp_get_wtime()`.
- Compute speedup, efficiency, and cost metrics.
- Understand strong vs weak scaling using Amdahl's and Gustafson's laws.
- Identify performance bottlenecks such as load imbalance, synchronization overhead, false sharing, and memory bandwidth saturation.
- Gain introductory exposure to performance profiling tools.

Experimental Environment

- Shared-memory multicore CPU.
- Operating System: Linux / Ubuntu (native or virtual machine).
- Compiler: gcc / g++ with OpenMP support.
- Number of threads: 1, 2, 4, 8 (or up to available cores).

A) Experiment 1: Execution Time Measurement: Objective: Measure the execution time of a parallel OpenMP loop using `omp_get_wtime()`.

Task:

1. Compile the given vector addition program with OpenMP support.
2. Run the program with one thread.
3. Run the program with multiple threads.
4. Record execution time.

Execution Time

```
double start = omp_get_wtime();
/* parallel region */
#pragma omp parallel for
for (long i = 0; i < N; i++)
    C[i] = A[i] + B[i];
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
double end = omp_get_wtime();  
printf("Time: %f\n", end - start);
```

T_1 : Run with OMP_NUM_THREADS=1

T_p : Run with OMP_NUM_THREADS=p

B. Experiment 2: Speedup and Efficiency

$$S(p) = \frac{T_1}{T_p}$$

Example:

Threads	Time (s)	Speedup
1	8.0	1.0
4	2.3	3.48
8	1.4	5.71

Throughput:

$$\text{Throughput} = \frac{N}{T_p}$$

Shows how many elements are processed per second.

Efficiency in OpenMP

Parallel Efficiency

$$E(p) = \frac{S(p)}{p}$$

Example (8 thread):

$$E(8) = \frac{5.71}{8} = 0.71$$

Good efficiency: (>70%)

C. Experiment 3: Strong Scaling vs Weak Scaling

Strong scaling refers to scenarios where the problem size stays the same while the number of threads increases, whereas weak scaling involves growing the problem size in direct proportion to the number of threads. The task is to analyze and compare execution times for both approaches, offering explanations for the observed performance patterns. *DO : Compare execution times and explain observed behavior*

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Interpretation: The relationship between thread count and execution time reveals fundamental performance constraints. If execution time remains steady despite adding more threads, the process is likely memory-bound, limited by data access speeds rather than computational power. Conversely, if execution time increases with additional threads, overhead costs like synchronization and load imbalance become the dominant bottleneck. These patterns help identify whether optimization efforts should focus on memory access patterns or parallelization efficiency.

D. Cost Metric:

$$C = p \times T_p$$

Eg: $T_1 = 8s$ $T_8 = 1.4s$

Cost = $8 \times 1.4 = 11.2$ (Not cost-optimal (ideal 8s))

8. Cache Miss Rate (Indirect)

Symptoms of high cache misses:

- Speedup flattens early
- Performance varies with schedule

```
#pragma omp parallel for schedule(static)
```

E. Experiment 4: Scheduling and Load Imbalance:

In OpenMP, the scheduling policy determines how loop iterations are distributed among the available threads, which is critical for maintaining load balance when the workload per iteration is not uniform.

1. Static Scheduling (schedule(static))

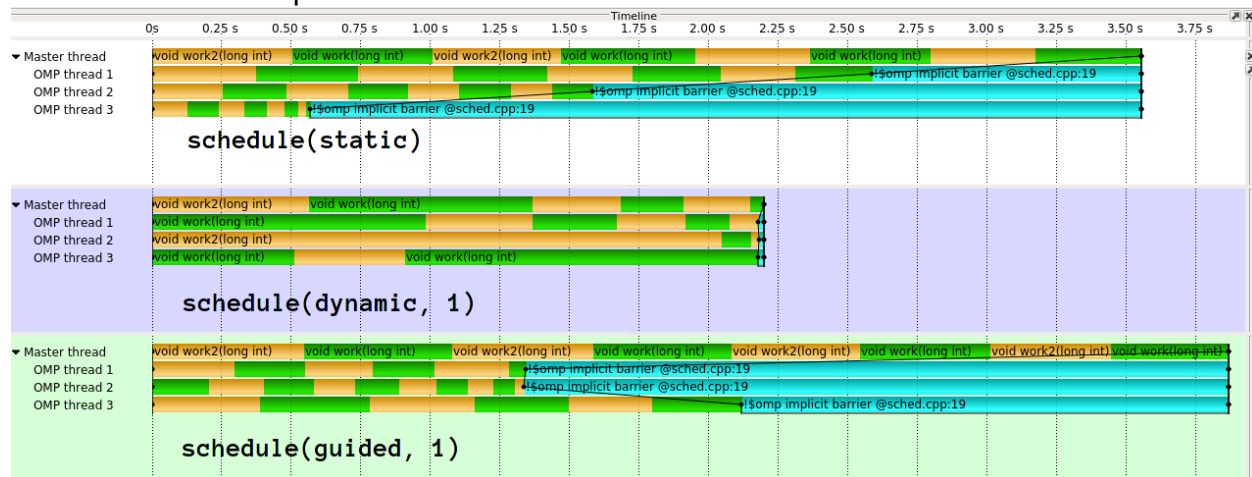
Static scheduling is the default policy where the total iteration space is divided into equal-sized chunks and assigned to threads at the very beginning of the loop (at compile or launch time). While this method has the lowest overhead because the distribution is pre-determined, it is highly susceptible to load imbalance. If some iterations take longer than others for example, in a Mandelbrot set calculation or a triangular matrix operation some threads will finish early and sit idle while others are still working. This results in poor CPU utilization, as the entire program must wait for the slowest "straggler" thread to finish its pre-assigned work.

2. Dynamic Scheduling (schedule (dynamic, chunk))

Dynamic scheduling addresses imbalance by treating the iteration space like a “work queue.” Each thread requests a small chunk of iterations, processes them, and then returns to the queue to ask for more. This ensures that faster threads or those assigned “easier” work do more of the total iterations, keeping all cores busy until the queue is empty. However, this flexibility comes at the cost of increased overhead, as threads must constantly synchronize to safely access and update the shared loop counter. Choosing a chunk size that is too small can lead to threads spending more time communicating than doing actual work.

3. Guided Scheduling (schedule(guided))

Guided scheduling is a sophisticated hybrid approach designed to minimize both imbalance and overhead. It starts by assigning large chunks to threads to reduce the initial number of synchronization requests. As the loop progresses and the remaining work decreases, the chunk size is exponentially reduced (down to a specified minimum). This allows the scheduler to “guide” the threads toward the end of the loop with finer granularity, ensuring that no thread is left with a massive, time-consuming block of work right at the finish line. It is often the most efficient choice for highly irregular workloads where the cost of iterations is unpredictable.



$$\text{Imbalance} = \frac{T_{\max} - T_{\text{avg}}}{T_{\text{avg}}}$$

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
double t_start = omp_get_wtime();
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    double local_start = omp_get_wtime();
    #pragma omp for
    for (int i = 0; i < N; i++)
        work(i);
    double local_end = omp_get_wtime();
    thread_time[tid] = local_end - local_start;
}
```

Use: export OMP_SCHEDULE="static" OR "dynamic,10" OR "guided"

Scheduling Policy Impact

Policy	Distribution Timing	Overhead	Best For...
Static	Compile/Launch Time	Lowest	Uniform workloads (e.g., simple vector addition).
Dynamic	Run Time (On- demand)	Highest	Highly irregular workloads with unpredictable costs.
Guided	Run Time (Adaptive)	Medium	Workloads that vary, but benefit from low initial overhead.

F. Experiment 5: Synchronization Overhead (critical,atomic, reduction)

This experiment evaluates the overhead of thread synchronization by comparing a manual mutual exclusion approach against a compiler-optimized collective operation. In the critical section implementation, threads are forced to serialize their access to a single shared sum variable, creating a significant bottleneck where N-1 threads must wait while one thread updates the value. Conversely, the reduction clause allows each thread to maintain a private partial sum, which is only

UCS645: PARALLEL & DISTRIBUTED COMPUTING

merged into the global total at the end of the loop using highly efficient tree-based hardware or software optimization. When comparing these methods, you will typically observe that the critical section fails to scale, often performing slower as more threads are added due to lock contention, whereas the reduction clause exhibits near-linear speedup and minimal performance degradation.

In parallel computing, Atomic operations are the “middle ground” between the heavy-handed Critical Section and the highly specialized Reduction. While a critical section uses a software lock that pauses the thread, an atomic operation leverages specific hardware instructions (like “Fetch-and-Add”) to update a variable in a single, uninterruptible step. This avoids the massive overhead of the OS managing locks.

The Performance Spectrum

If you were to rank them based on your recent experiments, the results typically look like this:

1. Reduction (Fastest): No synchronization during the loop. Each thread works in total isolation.
2. Atomic (Middle): Threads “fight” for the same memory address, but they do it using fast hardware-level signals. It is much faster than critical but still slower than reduction because of the “memory contention” (threads waiting for the memory bus).
3. Critical (Slowest): Threads use a heavy software lock. The CPU must stop, check the lock status, and potentially put the thread to sleep if the lock is held.

G: Experiment 6: False Sharing (OpenMP-Specific):

```
int partial_sum[NUM_THREADS]; // shared cache line
```

Sol:

```
int partial_sum[NUM_THREADS][CACHE_LINE/sizeof(int)];
```

This experiment explores the phenomenon of false sharing, a subtle performance trap where multiple CPU cores inadvertently conflict by updating different variables that happen to reside on the same cache

UCS645: PARALLEL & DISTRIBUTED COMPUTING

line (typically 64 bytes). In the unpadded implementation, threads writing to adjacent array elements force the hardware to constantly synchronize and invalidate caches across cores, creating a "ping-pong" effect that drastically slows execution despite no logical dependency between the data. By applying cache-line padding—inserting dummy data to ensure each thread's variable is isolated on its own unique cache line—we eliminate this hardware-level contention. Comparing the two versions will typically reveal a dramatic speedup in the padded version, as it restores true parallel efficiency by allowing each core to operate on its local cache without interference.

Metric

- Cache miss rate
- CPI increase

Note: to show you the perf commands to specifically track L1-dcache-load-misses:

```
sudo perf stat -e L1-dcache-load-misses,L1-dcache-stores,cycles,instructions ./eg14
```

In WSL/ Virtual box:

```
sudo /usr/lib/linux-tools/6.8.0-90-generic/perf stat -e L1-dcache-load-misses,L1-dcache-stores,cycles,instructions ./eg15
```

H: Experiment 7: Memory Bandwidth Saturation

This experiment investigates the hardware limitations of multicore scaling by measuring effective memory bandwidth during a Triad kernel operation ($A = B + \alpha \times C$). Using the formula

Memory Bandwidth Utilization: The effective bandwidth is calculated based on the total data moved across the memory bus divided by the time it took to move it:

$$\text{Bandwidth} = \frac{3 \times N \times \text{sizeof}(\text{double})}{T_p}$$

3: Represents the three memory accesses (A, B, and C).

N: The number of elements in the vectors.

sizeof(double): Usually 8 bytes.

UCS645: PARALLEL & DISTRIBUTED COMPUTING

T_p : The parallel execution time measured in seconds.

I. Barrier Cost

```
#pragma omp barrier
```

Note : Threads wait for slowest thread and Excess barriers reduce scalability.

J. Energy & Practical Metrics

$$\text{Energy-to-Solution } E = P \times T_p$$

Note: More threads → higher power but lower execution time.

Threads	Time (s)	Speedup	Efficiency	Bandwidth (GB/s)
1	8.0	1.0	1.00	5.1
4	2.3	3.48	0.87	18.2
8	1.4	5.71	0.71	21.4

DIY: OpenMP Experiment

Measuring Speedup & Efficiency (Vector Addition)

Objective: Measure execution time, speedup, and efficiency for an OpenMP

loop. [eg1.cpp]

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main() {
    long N = 100000000;
    double *A = malloc(N*sizeof(double));
    double *B = malloc(N*sizeof(double));
    double *C = malloc(N*sizeof(double));

    for (long i=0;i<N;i++) A[i]=B[i]=1.0;
    double start = omp_get_wtime();
    #pragma omp parallel for
    for (long i=0;i<N;i++)
        C[i] = A[i] + B[i];
    double end = omp_get_wtime();
    printf("Time = %f seconds\n", end-start);
    return 0;
}
```

```
gcc -O3 -fopenmp eg1.c -o eg1
```

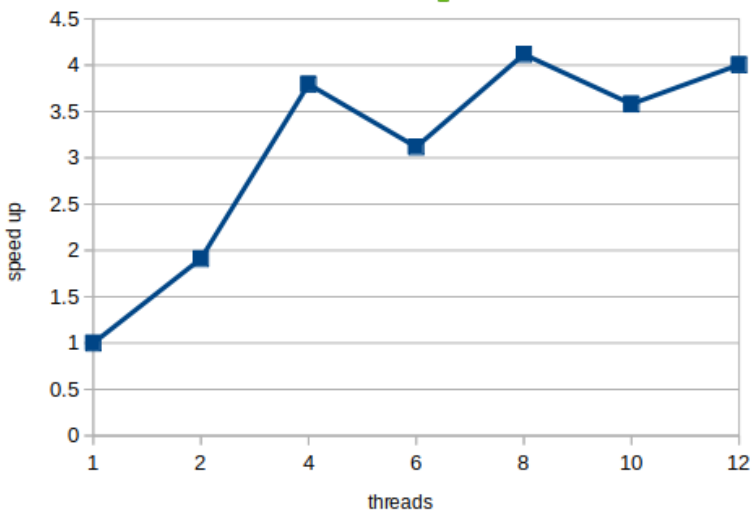
```
OMP_NUM_THREADS=1 ./eg1
```

```
OMP_NUM_THREADS=8 ./eg1
```

Metrics

UCS645: PARALLEL & DISTRIBUTED COMPUTING

$$S(p) = \frac{T_1}{T_p} \quad E(p) = \frac{S(p)}{p}$$



Strong Scaling vs Weak Scaling [eg2.cpp]

For **Strong Scaling**: Fix **N** and increase the number of threads

Weak Scaling: `long N = 10000000 * omp_get_max_threads();`

Compute

Amdahl's Law: The Pessimist's Bottleneck

Proposed by Gene Amdahl in 1967, this law focuses on **fixed-size workloads**. It asks: "If I have a specific task, how much faster will it go if I add more processors?"

The "catch" is the serial fraction—the part of the code that simply cannot be parallelized (like booting up or final data merging). No matter how many processors you throw at it, that serial part will always take the same amount of time.

The Formula

The speedup S is calculated as:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

- **P**: The proportion of the program can be made parallel.
- **1 - P**: The serial proportion.
- **N**: The number of processors.

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Amdahl's Law predicts **diminishing returns**. As N goes to infinity, the maximum speedup is stuck at $\frac{1}{1-P}$. If 10% of code is serial, it can never go faster than 10x, even with a million processors.

Gustafson's Law: The Optimist's Horizon

John Gustafson countered in 1988 by arguing that we don't usually keep the problem size fixed when we get more power. Instead, we use that extra power to **solve bigger, more complex problems** in the same amount of time. This is known as **scaled speedup**. It assumes that as you add processors, the parallel part of the workload grows, while the serial part stays constant.

The Formula

The speedup S is calculated as:

$$S = N + (1 - N)(1 - P) = (1 - P) + N \times P$$

P = The proportion of the program that is parallel.

N : The number of processors.

Gustafson's Law suggests a **linear relationship**. It's much more optimistic because it suggests that as long as you have more data to process, adding more processors will continue to give you significant benefits.

Differences:

Feature	Amdahl's Law	Gustafson's Law
Perspective	Fixed Workload	Scaled Workload
Goal	Minimize time for a specific task.	Maximize work done in a fixed time.
Scaling	Problem size is constant.	Problem size increases with processors.
View of Serial Code	A hard "ceiling" on speed.	A manageable constant cost.
Real-world use	Personal computing, short tasks.	Supercomputing, Big Data, Weather modeling.

Strong Scaling & Amdahl's Law

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Strong scaling is the direct application of Amdahl's Law. It asks: *How does the time to solution vary with the number of processors for a **fixed** total problem size?*

- **The Goal:** Speed. You want to get the same result as fast as humanly (or electronically) possible.
- **The Constraint:** You are hitting a "wall." Because the workload is fixed, as you add more processors, the portion of the task that can't be shared (serial overhead) eventually dominates the execution time.
- **Real-world Example:** Compiling a specific piece of software. Adding more cores helps, but eventually, the time it takes to link the files (a serial task) means adding a 100th core provides zero benefit.

Weak Scaling & Gustafson's Law

Weak scaling is the practical embodiment of Gustafson's Law. It asks: *How does the time to solution vary with the number of processors for a **fixed problem size per processor?***

- **The Goal:** Throughput and Resolution. You keep the amount of work on each individual core the same, which means the total problem size grows as you add hardware.
- **The Constraint:** If you double the processors and double the data, and the execution time stays the same, you have "perfect" weak scaling.
- **Real-world Example:** Weather forecasting. Instead of trying to predict today's weather faster (Strong Scaling), scientists use more processors to predict the weather with much higher resolution—looking at 1km squares instead of 10km squares—in the same amount of time.

Feature	Strong Scaling	Weak Scaling
Mathematical Basis	Amdahl's Law	Gustafson's Law

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Total Workload	Fixed / Constant	Increases with N
Work per Processor	Decreases as N increases	Remains Constant
Primary Metric	Reduction in execution time.	Increase in work capacity.
The "Failure" State	Speedup plateaus (Amdahl's bottleneck).	Execution time increases as data grows.

Which one should you use?

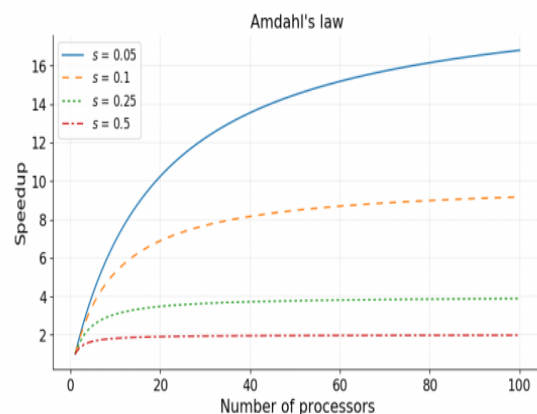
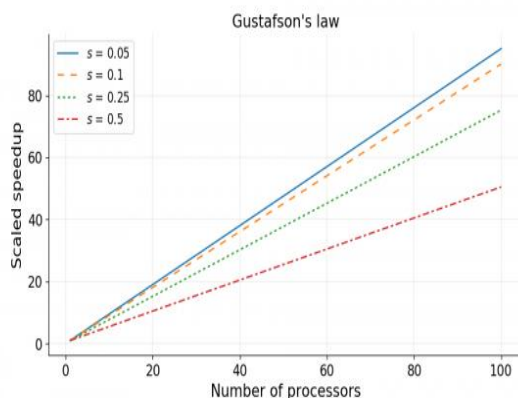
It depends on your hardware and your problem.

- Use **Strong Scaling** if you have a deadline you need to meet (e.g., "This simulation must finish by 5 PM"). It tells you when you've reached the point of wasting money on extra cores.
- Use **Weak Scaling** if you are trying to push the boundaries of what is possible (e.g., "I want to simulate the entire galaxy, not just one solar system"). It tells you how well your code handles "Big Data."

Most developers use **Strong Scaling** to find the limits of their code's efficiency and **Weak Scaling** to justify the need for a larger supercomputer cluster.

Observation:

Strong scaling can result in a memory bottleneck, while weak scaling often leads to improved efficiency.



Eg:

```
// example of strong and weak scaling in C++ using OpenMP
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
// measuring performance of Pi calculation using numerical integration
// as we vary the number of threads and problem size
#include <iostream>
#include <omp.h>
#include <chrono>
#include <vector>
#include <iomanip>
using namespace std;
// Performance measurement function
double calculate_pi_parallel(long long steps, int num_threads) {
    double step = 1.0 / (double)steps;
    double sum = 0.0;
    auto start = high_resolution_clock::now();
    #pragma omp parallel num_threads(num_threads)
    {
        double x, local_sum = 0.0;
        #pragma omp for
        for (long long i = 0; i < steps; i++) {
            x = (i + 0.5) * step;
            local_sum += 4.0 / (1.0 + x * x);
        }
        #pragma omp atomic
        sum += local_sum;
    }
    auto end = high_resolution_clock::now();
    duration<double> diff = end - start;
    return diff.count();
}

int main() {
    int max_cores = 23;
    long long base_steps = 100000000; // 100 Million steps
    cout << fixed << setprecision(4);
    // --- SECTION 1: STRONG SCALING ---
    // Problem size is FIXED at 500M steps
    long long strong_total_steps = 500000000;
    cout << "--- STRONG SCALING (Fixed Total Work: " << strong_total_steps << ") ---" << endl;
    cout << left << setw(12) << "Cores" << setw(15) << "Time (s)" << "Speedup" << endl;
    double t_serial_strong = calculate_pi_parallel(strong_total_steps, 1);
    cout << setw(12) << "1 (Serial)" << setw(15) << t_serial_strong << "1.00x" << endl;
    for (int n = 4; n <= max_cores; n += 4) {
        double p_time = calculate_pi_parallel(strong_total_steps, n);
        cout << setw(4) << n << " Cores" << setw(5) << ""
            << setw(15) << p_time << (t_serial_strong / p_time) << "x" << endl;
    }
    cout << "\n";
    // --- SECTION 2: WEAK SCALING ---
    // Problem size grows with core count (100M steps PER CORE)
    cout << "--- WEAK SCALING (Fixed Work Per Core: " << base_steps << ") ---" << endl;
    cout << left << setw(12) << "Cores" << setw(15) << "Total Work" << setw(15) << "Time (s)"
    << "Efficiency" << endl;
    double t_serial_weak = calculate_pi_parallel(base_steps, 1);
    cout << setw(12) << "1 (Serial)" << setw(15) << base_steps << setw(15) << t_serial_weak <<
    "100%" << endl;
    for (int n = 4; n <= max_cores; n += 4) {
        long long current_work = base_steps * n;
        double p_time = calculate_pi_parallel(current_work, n);
        // Efficiency = (T1 / Tn) * 100. In ideal weak scaling, Tn remains equal to T1.
        double efficiency = (t_serial_weak / p_time) * 100.0;
        cout << setw(4) << n << " Cores" << setw(5) << ""
            << setw(15) << current_work << setw(15) << p_time << efficiency << "%" << endl;
    }
}
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
    return 0;  
}
```

```
g++ -O3 -fopenmp eg2.cpp -o eg2
```

What you will observe on 8-23 cores:

1. **Amdahl's Reality:** As you move from 1 to 8 cores, you'll see a nice jump in speed. But as you approach 23, the speedup won't be 23X. It might only be 12X or 15X. This is Amdahl's Law in action: the time it takes to create threads and merge the results becomes a larger percentage of the total work.
2. **The "23-Core" Quirk:** If your CPU has **P-cores** (Performance) and **E-cores** (Efficiency), like modern Intel chips, you will see the speedup "bend" once you start using the E-cores. They aren't as fast as the P-cores, so they actually make the "Serial Fraction" $(1-P)$ appear larger than it really is.
3. **Memory Bottlenecks:** With 23 threads all trying to access your RAM at once, you might hit a "memory wall" where adding more cores actually makes the program *slower* because they are fighting over the same data bus.

Scheduling & Load Imbalance [eg3.cpp, eg11.cpp]

```
// Experiment: Static vs. Dynamic Scheduling to Handle Load Imbalance  
// Description: This code demonstrates how static scheduling can lead to load imbalance  
// in parallel loops with varying workloads, and how dynamic scheduling can help mitigate this  
// issue.  
// The workload increases with the loop index, simulating a scenario where  
#include <iostream>  
#include <vector>  
#include <omp.h>  
#include <chrono>  
#include <thread>  
#include <cmath>  
#include <string>  
  
using namespace std;  
using namespace std::chrono;  
// Simulates work that gets progressively harder  
void heavy_work(int iterations) {  
    // Artificial delay to simulate complex math  
    // Lower iterations = fast, Higher iterations = slow  
    double dummy = 0;  
    for (int i = 0; i < iterations * 1000; ++i) {  
        dummy += sin(i) * cos(i);  
    }  
}
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
void run_test(string label, bool use_dynamic) {
    int num_tasks = 1000;
    auto start = high_resolution_clock::now();
    // The "schedule" clause is the key here
    if (use_dynamic) {
        // Chunk size of 10 allows cores to grab small batches of work as they become free
        #pragma omp parallel for schedule(dynamic, 10)
        for (int i = 0; i < num_tasks; i++) {
            heavy_work(i); // Workload increases as i increases
        }
    } else {
        // Divide tasks equally (1000 / 23) regardless of how hard they are
        #pragma omp parallel for schedule(static)
        for (int i = 0; i < num_tasks; i++) {
            heavy_work(i);
        }
    }
    auto end = high_resolution_clock::now();
    duration<double> diff = end - start;
    cout << label << " Time: " << diff.count() << "s" << endl;
}

int main() {
    cout << "Testing Load Imbalance on " << omp_get_max_threads() << " cores..." << endl;
    cout << "-----" << endl;
    // 1. Static: Cores 0-5 get the easy start, higher cores get the brutal end.
    // Result: Most cores will finish in milliseconds and wait at the barrier for the last
    // core.
    run_test("Static Scheduling (Fixed Chunks) ", false);
    // 2. Dynamic: Faster cores help finish the heavy tasks at the end.
    // Result: Higher CPU utilization across all 23 cores.
    run_test("Dynamic Scheduling (Work Stealing)", true);
    return 0;
}
```

Why the Results Differ

When you run this on your machine, you will likely see that Dynamic Scheduling is significantly faster, even though it has a tiny bit more “management overhead.”

1. Static Scheduling (The “Fair” approach that fails)

With 23 cores and 1000 tasks, the compiler gives roughly 43 tasks to each core.

- Core 0 gets tasks 0-42 (Very easy, finishes instantly).
- Core 22 gets tasks 957-1000 (Very hard, takes a long time).
- Result: Core 0 sits idle for 90% of the run while Core 22 struggles alone. This is a massive Load Imbalance.

2. Dynamic Scheduling (The “Efficient” approach)

Tasks are handed out in small chunks (in the code, chunk size = 10).

- Cores 0 through 21 finish their "easy" early tasks quickly.

UCS645: PARALLEL & DISTRIBUTED COMPUTING

- Instead of going to sleep, they see there are still tasks left in the 900+ range.
- They “help” Core 22 by taking over those heavy tasks.
- Result: All 23 cores stay busy until the very end.

When to use which?

Scheduling Type	Best Used When...	Downside
Static	Each loop iteration takes exactly the same time.	Causes idle cores if work is unpredictable.
Dynamic	Work is unpredictable or “front-loaded/back-loaded.”	Small overhead for managing the task queue.
Guided	A mix: starts with large chunks and gets smaller.	Complex to tune the “chunk size” perfectly.

Note : If you are using **Hyper-threading** (where 1 physical core acts as 2 logical cores), Load Imbalance is even more common. Physical cores and logical threads don’t share cache equally. If you notice one thread “stalling,” switching to `schedule(guided)` or `schedule(dynamic)` is often the fastest way to gain a 20-30% performance boost without changing your actual logic.

Run: [eg4.cpp]

```
// Experiment: Measuring Load Imbalance with Different OpenMP Scheduling Strategies
// Description: This code demonstrates how different OpenMP scheduling strategies
// affect load imbalance in parallel loops with varying workloads.
// The workload increases with the loop index, simulating a scenario where
// some iterations take significantly longer than others.
// C++ Code: Load Imbalance Measurement with Various Schedules
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>
#include <numeric>
#include <algorithm>
#include <iomanip>
#include <cmath>
using namespace std;
using namespace std::chrono;
// A workload that gets progressively harder
void work(int i) {
    double dummy = 0;
    // Workload is proportional to i to create imbalance
    long long limit = (i + 1) * 200000;
    for (long long j = 0; j < limit; ++j) {
```


UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
        dummy += sin(j) * cos(j);
    }
}

void measure_imbalance(string schedule_name, int n_threads, int N) {
    vector<double> thread_times(n_threads, 0.0);
    auto start_wall = high_resolution_clock::now();
    // We use a manual parallel block to track time per thread
    #pragma omp parallel num_threads(n_threads)
    {
        int tid = omp_get_thread_num();
        auto start_thread = high_resolution_clock::now();
        // The specific loop we are testing
        if (schedule_name == "static") {
            #pragma omp for schedule(static)
            for (int i = 0; i < N; i++) work(i);
        } else if (schedule_name == "dynamic,4") {
            #pragma omp for schedule(dynamic, 4)
            for (int i = 0; i < N; i++) work(i);
        } else if (schedule_name == "guided") {
            #pragma omp for schedule(guided)
            for (int i = 0; i < N; i++) work(i);
        }
        auto end_thread = high_resolution_clock::now();
        duration<double> diff = end_thread - start_thread;
        thread_times[tid] = diff.count();
    }
    auto end_wall = high_resolution_clock::now();
    duration<double> wall_time = end_wall - start_wall;
    // Calculate Metrics
    double t_max = *max_element(thread_times.begin(), thread_times.end());
    double t_sum = accumulate(thread_times.begin(), thread_times.end(), 0.0);
    double t_avg = t_sum / n_threads;
    double imbalance = (t_max - t_avg) / t_avg;
    // Output results
    cout << left << setw(15) << schedule_name
         << setw(12) << t_max << "s"
         << setw(12) << t_avg << "s"
         << fixed << setprecision(2) << (imbalance * 100) << "%" << endl;
}

int main() {
    int N = 2000; // Total iterations
    int n_threads = omp_get_max_threads();
    cout << "Running on " << n_threads << " threads...\n";
    cout << left << setw(15) << "Schedule"
         << setw(13) << "T_max"
         << setw(13) << "T_avg"
         << "Imbalance (%)" << endl;
    cout << string(55, '-') << endl;
    measure_imbalance("static", n_threads, N);
    measure_imbalance("dynamic,4", n_threads, N);
    measure_imbalance("guided", n_threads, N);
    return 0;
}
```

Run this on `schedule(static)`, `schedule(dynamic, 4)`, `schedule(guided)`

Metric:

$$Imbalance = \frac{T_{max} - T_{avg}}{T_{avg}}$$

UCS645: PARALLEL & DISTRIBUTED COMPUTING

The **Imbalance** metric tells you how much "theoretical" speedup you are leaving on the table.

- If your Imbalance is **0.50 (50%)**, it means your slowest thread is taking 50% longer than the ideal fair share.
- To relate this back to **Amdahl's Law**: Load imbalance acts like an "artificial serial fraction." Even if your code is 100% parallelizable, if one thread takes twice as long as the others, your code effectively behaves as if it's partially serial!

Reduction vs. Critical:[eg5.cpp]

This is the battle between **Reduction** (efficient) and **Critical** (expensive).

```
// Experiment: Comparing Synchronization Methods in C++ with OpenMP
// Description: This code compares the performance of two common synchronization
// methods in OpenMP: critical sections and reduction clauses.
// The critical section method serializes access to a shared variable,
// while the reduction method allows threads to maintain private copies
// and combines them at the end, reducing contention.
// C++ Code: Synchronization Methods Comparison
#include <iostream>
#include <omp.h>
#include <chrono>
#include <iomanip>
using namespace std;
using namespace std::chrono;
void test_sync_methods(long long N) {
    double sum_critical = 0.0;
    double sum_reduction = 0.0;
    // 1. CRITICAL SECTION (The "One-at-a-time" approach)
    // Threads must wait for a software lock to update the global variable.
    auto start = high_resolution_clock::now();
    #pragma omp parallel for
    for (long long i = 0; i < N; i++) {
        #pragma omp critical
        {
            sum_critical += 1.0;
        }
    }
    auto end = high_resolution_clock::now();
    duration<double> time_critical = end - start;
    // 2. REDUCTION (The "Local-sum" approach)
    // Each thread gets a private copy of the variable and merges at the end.
    start = high_resolution_clock::now();
    #pragma omp parallel for reduction(+:sum_reduction)
    for (long long i = 0; i < N; i++) {
        sum_reduction += 1.0;
    }
    end = high_resolution_clock::now();
    duration<double> time_reduction = end - start;
    // Results
    cout << left << setw(20) << "Method" << "Time (s)" << endl;
    cout << "-----" << endl;
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
cout << setw(20) << "Critical Section" << time_critical.count() << "s" << endl;
cout << setw(20) << "Reduction" << time_reduction.count() << "s" << endl;
if (time_reduction.count() > 0) {
    cout << "\nOverhead Factor: " << (time_critical.count() / time_reduction.count())
        << "x slower" << endl;
}
}
int main() {
    // 10 Million iterations
    test_sync_methods(10000000);
    return 0;
}
```

Understanding the Metrics

1. Synchronization Overhead

This is the "tax" you pay for managing parallelism. It is the time spent on locks, thread creation, and communication rather than the actual math.

Metric Formula:

$$Overhead = T_{parallel} - \left(\frac{T_{serial}}{N}\right)$$

If your overhead is high, your $T_{parallel}$ will be much larger than the ideal $\frac{T_{serial}}{N}$. In the critical example above, the overhead is nearly 99% of the execution time.

2. Barrier Wait Time

An OpenMP loop has an **implicit barrier** at the end. No thread can move past the } of the loop until **every** thread has finished its work.

Barrier Wait Time for a specific thread i is:

$$T_{wait,i} = T_{max_thread} - T_i$$

- T_{max_thread} : The time taken by the slowest thread.
- T_i : The time thread i actually spent working.

The "Cost of Wait" Analysis

On your 23-core PC, if you have a Load Imbalance (as we discussed previously), the Barrier Wait Time becomes your biggest bottleneck.

Example: If 22 cores finish in 1ms, but core #23 takes 100ms because it got a "heavy" task, the total time for that block is 100ms. Your Synchronization Cost is the sum of all those 22 cores sitting idle at the barrier.

Feature	Critical Section	Reduction	Barrier
---------	------------------	-----------	---------

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Type	Active Locking	Local Aggregation	Global Sync
Cost	Extremely High (Serializes)	Very Low	Depends on Imbalance
Scalability	Poor (Gets worse with more cores)	Excellent	Depends on workload
Use Case	Updating a file or complex object	Summing, Min/Max, products	Ensuring data is ready for next step

Learn about : how to use `omp_get_wtime()` to measure the exact idle time (barrier wait) of each of your 23 threads individually.

False Sharing Detection [eg6.cpp]: False sharing is one of the most silent and frustrating performance killers on high-core-count machines like yours. It occurs when two or more processors are modifying different variables that happen to reside on the same Cache Line (usually 64 bytes). Even though the threads aren't technically sharing data, the CPU's "Cache Coherency" protocol thinks they are. Every time Core 1 updates its variable, it invalidates the cache line for Core 2, forcing Core 2 to reload it from memory. This leads to a massive spike in Cache Misses and CPI.

```
// example of false sharing and its mitigation using padding in C++ with OpenMP
// This code demonstrates the performance impact of false sharing
// and how padding can be used to mitigate it.
// False sharing occurs when threads on different processors modify variables
// that reside on the same cache line, leading to performance degradation.
// This example compares performance with and without padding.
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>
using namespace std;
using namespace std::chrono;
// 64 bytes is the standard cache line size for most modern CPUs
#define CACHE_LINE_SIZE 64

struct BadStruct {
    long long val; // Only 8 bytes
};
struct PaddedStruct {
    long long val; // 8 bytes
    // We add 56 bytes of "junk" to ensure no two 'val' fields sit on the same cache line
    char padding[CACHE_LINE_SIZE - sizeof(long long)];
};
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
};  
void run_test(bool use_padding, long long iterations) {  
    int n_threads = omp_get_max_threads();  
    vector<BadStruct> bad_data(n_threads, {0});  
    vector<PaddedStruct> good_data(n_threads, {0});  
  
    auto start = high_resolution_clock::now();  
  
    #pragma omp parallel num_threads(n_threads)  
    {  
        int tid = omp_get_thread_num();  
        for (long long i = 0; i < iterations; i++) {  
            if (use_padding)  
                good_data[tid].val += 1;  
            else  
                bad_data[tid].val += 1;  
        }  
    }  
    auto end = high_resolution_clock::now();  
    duration<double> diff = end - start;  
  
    cout << (use_padding ? "Padded (Correct Mitigation): " : "False Sharing (The Problem): ")  
          << diff.count() << "s" << endl;  
}  
  
int main() {  
    long long iterations = 100000000; // 100 Million  
    cout << "Testing False Sharing on " << omp_get_max_threads() << " threads" << endl;  
    cout << "-----" << endl;  
    run_test(false, iterations); // High cache contention  
    run_test(true, iterations);  // Independent cache lines  
  
    return 0;  
}
```

The Metrics of False Sharing

When you run this on 23 cores, the “Bad” version might be **10x to 50x slower** than the “Good” version. To understand why, we look at two hardware metrics.

1. Cache Miss Rate (L1/L2 Invalidation)

In a healthy parallel program, each core should find its data in its local L1 cache (a “Cache Hit”). In False Sharing:

- **The Symptom:** Your “Remote Invalidation” or “L1 Miss” count skyrockets.
- **The Metric:**

$$\text{Miss Rate} = \frac{\text{cache miss}}{\text{Total Memory Access}} \times 100$$

UCS645: PARALLEL & DISTRIBUTED COMPUTING

- **On your PC:** You would see the CPU cores “fighting” for ownership of the same memory address, even though they are writing to different offsets.

2. CPI (Cycles Per Instruction) Increase

CPI measures how many CPU clock cycles it takes to execute a single instruction.

- **Ideal Parallel CPI:** Should be low (e.g., 0.5 to 1.5).
- **False Sharing CPI:** Can jump to **20.0 or higher**.
- **Why:** The CPU pipeline stalls completely. It cannot execute the next increment instruction because it is waiting for the Cache Coherency hardware to fetch the “latest” version of the cache line from a different core's cache.

Example Scenario: 23 Cores Scaling

If you were to graph the performance of the “Bad” code from 1 to 23 cores, you would see a paradoxical result:

Core Count	Result	Why?
1 Core	Fast	No contention.
2 Cores	Slower	The two cores start “ping-ponging” the cache line.
23 Cores	Disastrous	23 cores are all trying to invalidate each other. The system spends more time moving cache lines than doing math.

How to Detect this on your Machine

Since you are likely on Linux or Windows:

1. **Linux (perf):** Run `perf stat -e L1-dcache-load-misses ./your_program`. You will see millions of misses in the “Bad” version and almost zero in the “Good” version.
2. **Windows (Intel VTune / AMD uProf):** Look for “Data Sharing” or “Contention” metrics. It will highlight the exact line of code causing the “Cache Line Contention.”

UCS645: PARALLEL & DISTRIBUTED COMPUTING

Memory Bandwidth Saturation [eg7.cpp]: Memory Bandwidth Saturation occurs when your CPU cores are so fast that they spend most of their time waiting for the "data bus" to bring information from the RAM. Even if you have 23 cores, they all share a limited number of memory channels (typically 2 or 4 on consumer PCs). When all 23 cores scream for data at once, they hit a "Memory Wall," and adding more cores provides zero speedup.

```
// example of strong scaling in C++ using OpenMP
// measuring performance of a simple vector operation ( $A = B + \text{scalar} * C$ )
// as we increase the number of threads
// to illustrate strong scaling behavior
// and memory bandwidth considerations
// Note: Ensure your system has enough memory to handle large arrays
// This is a memory-bound operation because very little math is
// done compared to the amount of data being moved.
// C++ Code: Vector Triad (The "Stream" Benchmark)
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>
#include <iomanip>
#include <string>

using namespace std;
using namespace std::chrono;

int main() {
    // Large enough to exceed L3 cache (~800MB per vector, ~2.4GB total)
    long long N = 100000000;
    vector<double> A(N, 0.0), B(N, 1.1), C(N, 2.2);
    double scalar = 3.3;

    int max_threads = omp_get_max_threads();

    cout << "Memory Bandwidth & Scalability Test (Triad Kernel)" << endl;
    cout << left << setw(10) << "Cores"
         << setw(15) << "Time (s)"
         << setw(15) << "BW (GB/s)"
         << "Speedup" << endl;
    cout << string(55, '-') << endl;

    double t_serial = 0;

    for (int threads = 1; threads <= max_threads; threads++) {
        auto start = high_resolution_clock::now();

        // The "Triad" operation:  $A = B + s * C$ 
        #pragma omp parallel for num_threads(threads)
        for (long long i = 0; i < N; i++) {
            A[i] = B[i] + scalar * C[i];
        }

        auto end = high_resolution_clock::now();
        duration<double> diff = end - start;
        double Tp = diff.count();
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
if (threads == 1) t_serial = Tp;

// Metric: 3 arrays (A, B, C) x N elements x 8 bytes per double
double total_data_gb = (3.0 * N * sizeof(double)) / 1e9;
double BW = total_data_gb / Tp;

cout << left << setw(10) << threads
      << setw(15) << Tp
      << setw(15) << BW
      << fixed << setprecision(2) << (t_serial / Tp) << "x" << endl;
}

return 0;
}
```

Explaining the Metric

The formula you provided calculates the **Effective Bandwidth**.

$$BW = \frac{3 \times N \times \text{sizeof}(\text{double})}{T_p}$$

3 X N: We are accessing three vectors (A, B, C). *Read B[i], Read C[i] Write A[i]* (In modern CPUs, this also involves a “Read-for-Ownership,” technically making it 4 accesses, but 3 is the standard benchmark metric).

T_p: The parallel execution time.

The Result: Usually expressed in **GB/s**.

Experiment DIY [eg16.cpp]:

```
// Experiment: Measuring Memory Bandwidth with OpenMP in C++
// Description: This code measures the memory bandwidth of a simple vector operation
// (A = B + alpha * C) using OpenMP for parallelization.
// It calculates the total data moved and computes the effective bandwidth in GB/s.
// C++ Code: Memory Bandwidth Measurement
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>
using namespace std;
using namespace std::chrono;
int main() {
    long N = 100000000; // 100 Million elements
    vector<double> A(N, 0.0), B(N, 1.0), C(N, 2.0);
    double alpha = 0.5;
    int threads = omp_get_max_threads();
    auto start = high_resolution_clock::now();
    #pragma omp parallel for num_threads(threads)
    for (long i = 0; i < N; i++) {
        A[i] = B[i] + alpha * C[i];
    }
    auto end = high_resolution_clock::now();
```


UCS645: PARALLEL & DISTRIBUTED COMPUTING

```
duration<double> elapsed = end - start;
// Total Data Moved: 2 Reads (B,C) + 1 Write (A) = 3 * N * 8 bytes
double total_bytes = 3.0 * N * sizeof(double);
double bandwidth_gb_s = (total_bytes / elapsed.count()) / 1e9;
cout << "Threads: " << threads << endl;
cout << "Time:      " << elapsed.count() << " s" << endl;
cout << "Bandwidth: " << bandwidth_gb_s << " GB/s" << endl;
return 0;
}
```

Even though you are virtualized, the Memory Wall still exists.

1. Run the code with `export OMP_NUM_THREADS=1`, then 2, then 4, then 8.
2. Observe: You will likely see that bandwidth increases from 1 to 2 threads, but stays almost the same from 4 to 8 threads.

The Speedup Plateau

As you run the code, you will likely see the following:

- **1 to 4 Cores:** Speedup increases almost linearly (1x, 2x, 3.8x).
- **8 to 23 Cores:** Speedup flattens out. You might find that 16 cores and 23 cores result in the exact same time.
- **The Reason:** Your RAM can only deliver, say, 50 GB/s. If 4 cores are already consuming 48 GB/s, adding a 5th, 6th, or 23rd core doesn't help. They simply spend more time stalled in a "Wait" state.

2. The Arithmetic Intensity (optional)

This experiment highlights **Arithmetic Intensity (AI)**:

$$AI = \frac{\text{Floating Point Operations}}{\text{Bytes Moved}}$$

DIY: Rewrite the loop using “**Tiling**” or “**Blocking**” to improve Cache Reuse and reduce the pressure on the memory bandwidth. To through the “Memory Wall,” we use a technique called **Cache Blocking (or Tiling)**.

Since bandwidth is saturated, the goal is to stop going to slow RAM so often. Instead of streaming through the whole array once, we break the data into "blocks" that fit entirely into CPU's L3 or L2 cache. This way, the cores can reuse the data multiple times without triggering a new memory request.

C++ Code: Cache Blocking Example

UCS645: PARALLEL & DISTRIBUTED COMPUTING

This example simulates a 2D grid operation (like an image filter or matrix math) where data reuse is possible.

Cheat sheet:

Bottleneck	Key Metric to Calculate	If Metric is High...	Fix/Optimization
Amdahl's Law	$S = \frac{1}{(1 - P) + \frac{P}{N}}$	Your serial "tax" is too high.	Move more logic into parallel blocks.
Load Imbalance	$Imbalance = \frac{T_{max} - T_{avg}}{T_{avg}}$	Some cores are idling while one struggles.	Use schedule(dynamic) or guided.
Sync Overhead	$Overhead = T_{parallel} - \left(\frac{T_{serial}}{N}\right)$	Too much time spent in locks/barriers.	Use reduction instead of critical.
False Sharing	CPI (Cycles Per Instruction)	Cores are fighting over the same cache line.	Use alignas(64) or data padding.
Memory Wall	$BW = \frac{3 \times N \times sizeof(double)}{T_p}$	You've maxed out your RAM's speed.	Use Cache Tiling or Blocking.

Profiling Guide (LIKWID or VTune (will not work in WSL or VB ubuntu , perf))

Install:

```
sudo apt install likwid
```

```
Basic Metrics : likwid-perfctr -C 0-7 -g FLOPS_DP ./eg2
```

```
Memory Bandwidth: likwid-perfctr -C 0-7 -g MEM ./eg2
```

Metric	Meaning
CPI	Cycles per instruction

UCS645: PARALLEL & DISTRIBUTED COMPUTING

L3 miss	Cache pressure
MEM BW	Memory saturation

Linux perf (Low-Level Hardware View)

```
gcc -fopenmp -g -O3 eg2.cpp -o eg2
```

```
perf stat ./eg2 ( check previous how to run in virtual box )
```

Detailed Counters:

```
perf stat -e cycles,instructions,cache-misses,branches ./eg2
```

```
in VB: sudo /usr/lib/linux-tools/6.*.*-generic/perf stat -e L1-  
dcache-load-misses,L1-dcache-stores,cycles,instructions ./eg16
```

Interpretation

A high number of cache misses indicates poor data locality. A low IPC value suggests that the system is limited by memory performance.

Intel VTune Profiler (Deep Analysis):

1. Set up the Repository

First, you need to add Intel's keys and the repository to your system so you can use apt.

Install required tools

```
sudo apt update
```

```
sudo apt install -y gpg wget
```

Download Intel's GPG key

```
wget -O- https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-INTEL-  
SW-PRODUCTS.PUB \
```

```
| gpg --dearmor | sudo tee /usr/share/keyrings/oneapi-archive-  
keyring.gpg > /dev/null
```

Add the repository to your sources list

```
echo "deb [signed-by=/usr/share/keyrings/oneapi-archive-keyring.gpg]  
https://apt.repos.intel.com/oneapi all main" \
```

```
| sudo tee /etc/apt/sources.list.d/oneAPI.list
```

UCS645: PARALLEL & DISTRIBUTED COMPUTING

2. Install VTune

```
sudo apt update
```

```
sudo apt install intel-oneapi-vtune
```

3. Set Environment Variables

VTune doesn't add itself to your "PATH" automatically. You need to "source" the variables before you can use it.

```
source /opt/intel/oneapi/vtune/latest/env/vars.sh
```

4. Enable Hardware Profiling (Crucial Step)

To detect things like **Cache Misses** and **CPI**, VTune needs access to your CPU's hardware performance counters. By default, Linux restricts this for security.

Run this command to allow VTune to collect hardware data:

```
sudo sysctl -w kernel.perf_event_paranoid=1
```

5. Launch and Run

```
vtune-gui
```

How to use it for your metrics:

1. **Microarchitecture Exploration:** Use this to find **CPI** and **False Sharing**. It will highlight code in red if it's causing cache contention.
2. **Memory Access Analysis:** Use this to see your **Memory Bandwidth (GB/s)** and verify if you are hitting the "Memory Wall."
3. **HPC Performance Characterization:** This is the best overview for your 23-core machine. It shows **CPU Utilization** and **OpenMP Efficiency** (Load Imbalance).

Command Line:

Using the VTune command line (vtune) is often better for 23-core systems because it avoids the overhead of a GUI while your program is under heavy load. This ensures your **Load Imbalance** and **CPI** metrics are as accurate as possible.

1. Basic Performance Snapshot

UCS645: PARALLEL & DISTRIBUTED COMPUTING

To get a high-level overview of how well your OpenMP code is scaling across your cores:

```
vtune -collect hpc-performance ./your_executable
```

What it shows: CPU utilization, GFLOPS, and **Memory Bandwidth** usage.

Why for you: It will immediately flag if your 23 cores are “underutilized.”

2. Hunting for False Sharing and Cache Issues

To find that high **CPI** or **Cache Miss Rate** we discussed:

```
vtune -collect uarch-exploration ./your_executable
```

What it shows: A breakdown of where CPU cycles are spent (e.g., "Front-End Bound" vs. "Memory Bound").

The "False Sharing" tell: Look for "**Core Bound -> Data Sharing**" in the report.

3. Measuring Memory Bandwidth Saturation

To see if you've hit the “Memory Wall”:

```
vtune -collect memory-access ./your_executable
```

What it shows: A real-time graph of **GB/s** consumption.

The "Plateau" check: It will show you the peak bandwidth of your DRAM and how close your code gets to it.

4. Analyzing Load Imbalance

To see exactly which of your 23 threads is the “slacker” at the barrier:

```
vtune -collect threading ./your_executable
```

The Metric: Look for “**Effective CPU Utilization**”.

Visualization: It provides a "Timeline" view. If you see a lot of **red** (Spin Time) at the end of a loop, that is your **Barrier Wait Time**.

5. Generating a Readable Report

After running any of the commands above, VTune creates a results folder (e.g., r000hs). To see a summary in your terminal without opening the GUI, run:

EXERCISE:

Question 1: Molecular Dynamics - Force Calculation

Problem: Implement parallel computation of Lennard-Jones potential forces in a molecular dynamics simulation. Given N particles in 3D space, calculate the total potential energy and forces acting on each particle.

Tasks:

1. Parallelize the nested loops using OpenMP
2. Handle race conditions in force accumulation
3. Implement reduction for total energy
4. Optimize load balancing
5. Add performance measurement

Question 2: Bioinformatics - DNA Sequence Alignment (Smith-Waterman)

Problem: Implement a parallel version of the Smith-Waterman local sequence alignment algorithm for comparing two DNA sequences.

Tasks:

1. Parallelize the scoring matrix computation
2. Handle anti-dependencies in the dynamic programming approach
3. Experiment with different scheduling strategies
4. Implement wavefront parallelization (advanced)

Question 3: Scientific Computing - Heat Diffusion Simulation

Problem: Simulate heat diffusion in a 2D metal plate using finite difference method with parallel OpenMP implementation.

Hints for Question 3:

1. **Data Dependency Analysis:** The heat equation update has dependencies only on previous time step, allowing parallelization of spatial loops
2. **Reduction Operations:** Use `reduction(+:total)` for accumulating total heat

UCS645: PARALLEL & DISTRIBUTED COMPUTING

3. **Memory Optimization:** Consider loop tiling/cache blocking for better cache utilization
4. **Race Conditions:** No race conditions exist as each grid point writes to unique location in `next_temperature`.
5. **Scheduling Strategies:** Experiment with `schedule(static)`, `schedule(dynamic, chunk_size)`, and `schedule(guided)`.

Profiling Tools (Exposure Level)

Use the following tools for observation and analysis:

- `perf stat` – to measure cycles, instructions, and cache misses
- LIKWID – to observe FLOPS and memory bandwidth

Analysis experiment tables and graphs and give discussion.